To be able to apply gradient descent, we need the cost function to be derivable, at least in the neighborhood of the points where we compute the gradient.

Moreover, it helps if we already know the exact formula for the function we would like to optimize, so that we can also express the partial derivatives with mathematical formulas and compute gradients exactly. If we don't have the definition for the function to optimize, however, we can always resort to the formal definition of derivatives as mathematical limits, and compute the gradient numerically, by explicitly evaluating the ratio between Δf and Δx_i for increasingly small increments of each of the coordinates of the problem space.

One question you might want to ask is: if we do have the definition of `f`, and it is derivable, why do we have to run an iterative optimization, can't we just find its exact minimum using calculus?

Well, that's of course possible, in theory; it is also doable, at least for low-dimensional spaces and for some kind of functions.

However, finding exact solutions becomes hard to automate, and even to compute, in high-dimensional spaces: the number of equations needed to analytically find the global minimum grows exponentially with the problem's size; moreover, these functions can have hundreds, thousands or even an infinite number of local minima (think, for example, about `sin(x+y)` or even `x*sin(y)`) and to automate the search of a global optimum, we'd need to have all those points checked.

In general, gradient descent works well when we have a chance to design a cost function that has either a global minimum or, at most, a few local minima (better if of approximately the same